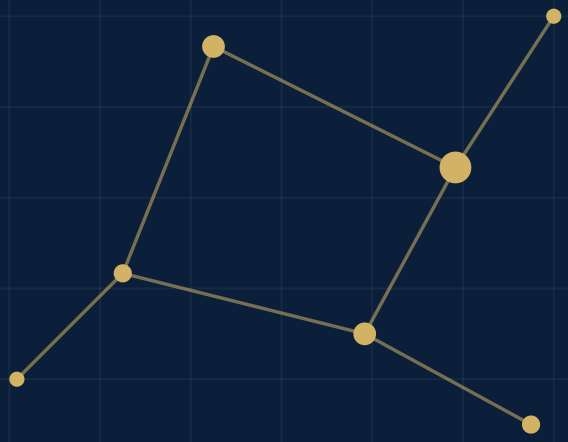


02 — DAY 1 · LIVE BUILD

CampusBite

Build an AI agent app step by step

Turn a plain starter into a campus canteen ordering agent using AI prompts in VS Code, then deploy it with a student screen and a live kitchen screen.

STACK	TOOLS	OUTPUT
Next.js · Supabase · Gemini	GitHub Copilot	Live app on Vercel

PREPARED BY

Sathish Kumar K.

AI Educator

linkedin.com/in/sathish-kumar-ceo

Learner handbook · Guide 2 of 3

November 2026

Contents

Build an AI agent app step by step. Follow the sections in order; each one ends with a checkpoint or checklist.

01	About this handbook	3
02	The problem statement	3
03	App design	4
04	Setup	5
05	Before you prompt: two things to know	8
06	Step 1: Plan with AI	9
07	Step 2: Read tools (the agent's eyes)	10
08	Step 3: Write tools (the agent's hands)	11
09	Step 4: System prompt (the agent's job description)	12
10	Step 5: API routes	14
11	Step 6: Student screen	14
12	Step 7: Kitchen screen	16
13	Step 8: Build, commit and deploy	17
14	Full test: six moments	18
15	Understand the agent loop	18
16	Stretch features	19
17	From CampusBite to your hackathon	20
18	Common errors and fixes	22
19	Key ideas to remember	23
20	Files and links	23

01 About this handbook

In this session you build **CampusBite**, an AI agent app for the campus canteen. You start from a plain starter project and, using AI prompts in VS Code, turn it into a working app with a student screen and a kitchen screen. Then you deploy it on the internet.

Follow the steps in order. Each step has a prompt to copy, what to check, and how to test it.

Part	What you do	Time
Setup	Your repo, settings, database and API keys	20 min
Steps 1 to 4	Plan, agent tools, system prompt	15 min
Steps 5 to 7	API routes, student screen, kitchen screen	15 min
Step 8	Build, commit and deploy	10 min
Full test	Try the six test moments on two screens	10 min
Stretch features	Add extra features with your own prompts	45 min
Hackathon planning	Plan your Day 2 problem the same way	30 min

You will learn: how to plan an agent app, the difference between read tools and write tools, why the system prompt matters, and why business rules belong in the database.

> **Before you start:** finish Guide 1, the Setup Guide (VS Code, Git, Node.js, GitHub Copilot and the free accounts).

02 The problem statement

Business user: a student who wants lunch between classes at the campus canteen. **Logged-in student:** `S001` (Priya Ramesh).

The problem today: long queues at the counter, popular items sell out before students reach the front, students do not know if their wallet balance is enough, and the kitchen cannot plan for the 1 pm rush.

The solution: students order in plain English. An AI agent checks the menu, stock, wallet and pickup time, then places the order. The kitchen sees it at once on a live screen.

User	What they do
Student (S001)	Asks for food in plain English, pays from the wallet, picks up at a chosen time
Kitchen staff	Sees new orders, marks them "Preparing", "Ready" and "Collected"

03 App design

Screens

Screen	What it shows
Student screen `/`	Agent chat (left), wallet and my orders, today's menu with live stock (right)
Kitchen screen `/kitchen`	Three columns: New orders, Preparing, Ready for pickup
Health check `/api/health`	Settings, connected Supabase project, database check

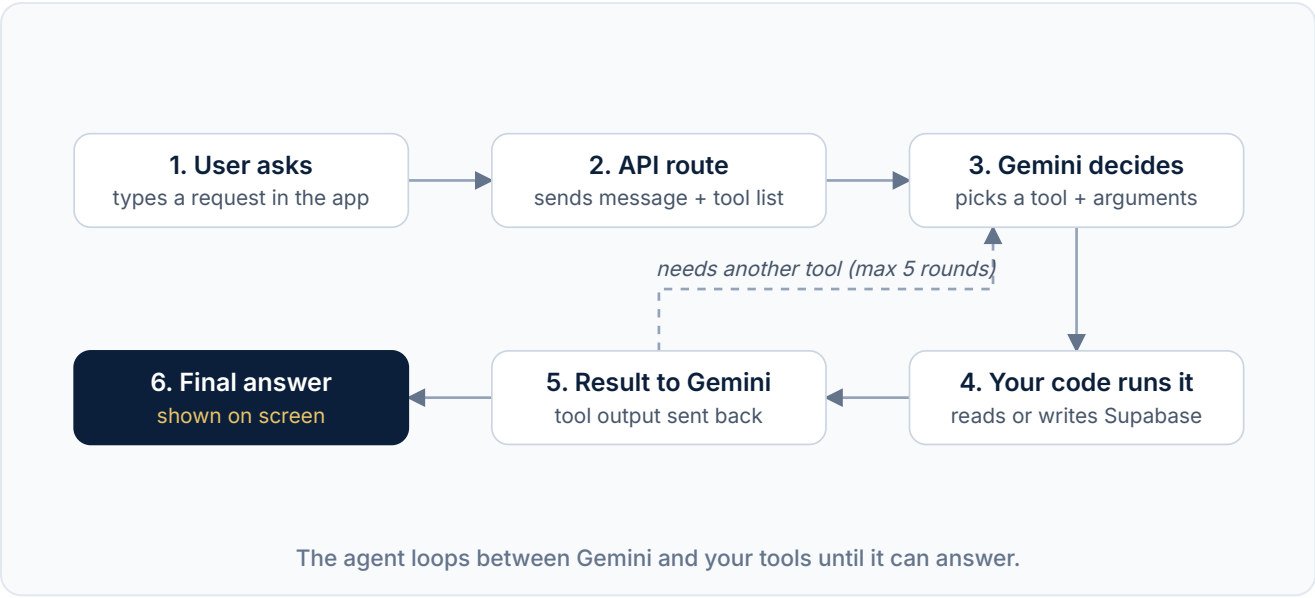
Data model (created by `seed/campusbite.sql`)

Table	Main columns
`menu_items`	id, name, category (meal, snack, drink, sweet), price, is_veg, stock
`wallets`	student_id, name, balance
`pickup_slots`	id, slot_time, capacity, booked (12:00 to 2:30 pm, every 15 minutes)
`orders`	id, student_id, slot_id, total, status (placed, preparing, ready, collected, cancelled)
`order_items`	order_id, menu_item_id, quantity, price

Starting data: S001 has Rs 150, Chicken Biryani has only 2 left, Paneer Butter Masala is sold out, the 12:30 slot is full and 1:00 is almost full. These help you test the rules.

Agent tools

Tool	Reads or writes	What it does
`getMenu`	Reads	Items in stock; filters for category, veg only, max price
`getWalletBalance`	Reads	The student's balance
`getPickupSlots`	Reads	Today's slots with free places
`getMyOrders`	Reads	The student's orders and status
`placeOrder`	Writes	Calls `place_order()`: order + items + wallet + stock + slot, all or nothing
`cancelOrder`	Writes	Calls `cancel_order()`: refund, stock back, slot freed (only before cooking)



The agent loops between Gemini and your tools until it can answer.

Key idea: `placeOrder` changes four things at once. The database functions check everything first (balance, stock, slot, maximum 5 items) and then make all changes together, or none. This is called a transaction. Because the rules live in the database, the AI cannot break them.

04 Setup

1. Create your own copy on GitHub

- Open `github.com/learn2simple/campusbite-live`
- Click **Use this template → Create a new repository**
- Name it `campusbite`, keep it **Public**, and click **Create repository**
- Make sure the GitHub account in your browser is the same one you use in VS Code (Accounts icon, bottom left)

2. Clone it outside OneDrive

TERMINAL

```
# Windows (PowerShell)
mkdir C:\Projects
cd C:\Projects

# Mac: use  mkdir ~/Projects  and  cd ~/Projects  instead

git clone <your repo HTTPS link>
cd campusbite
npm install
code .
```

If VS Code asks **Do you trust the authors?**, click **Yes**.

3. Create your settings file `.env.local`

```
TERMINAL

cp .env.example .env.local
```

Open `.env.local` (not `.env.example`) and fill in the values. No spaces or quotes.

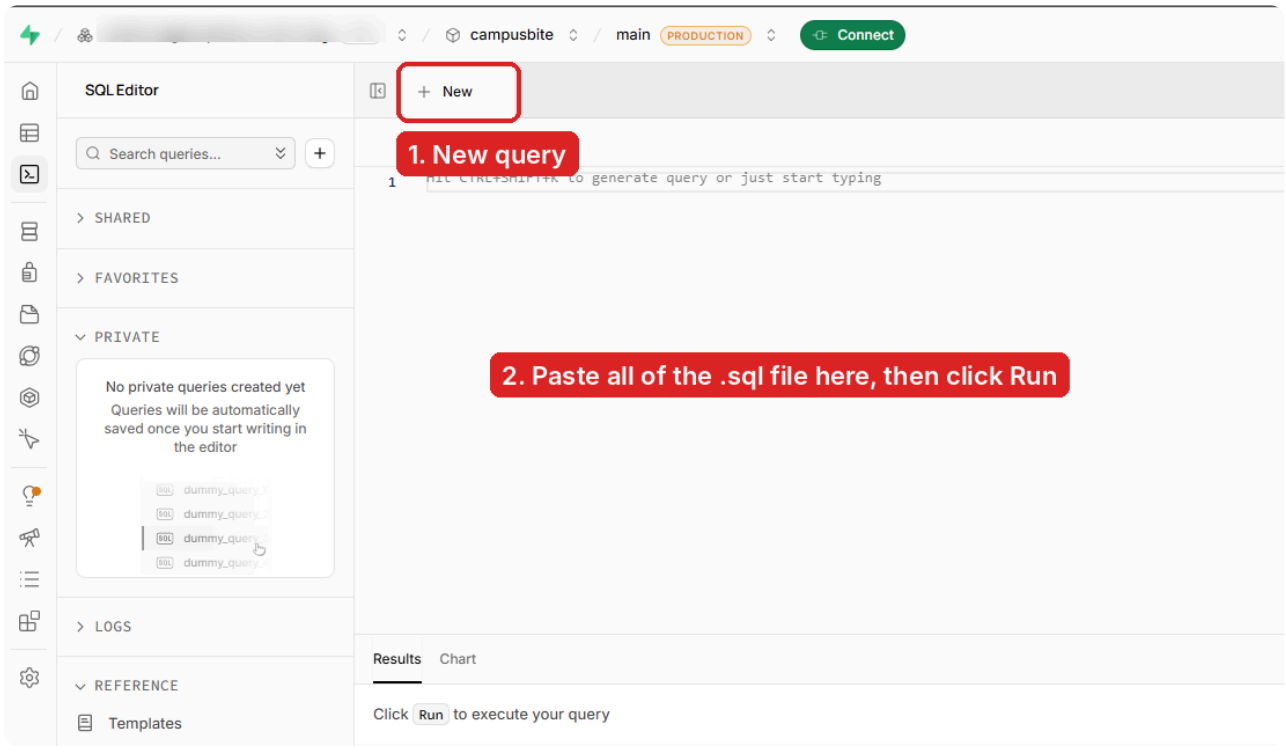
```
TERMINAL

GEMINI_API_KEY=your Gemini key
GEMINI_MODEL=gemini-3.8-flash
SUPABASE_URL=https://<project-id>.supabase.co
SUPABASE_SECRET_KEY=the secret key from the same project
APP_PASSCODE=pick-a-passcode
```

> **Keep the secret key private.** Real keys go only in `.env.local`. Never type them into `.env.example`, because that file goes to GitHub.

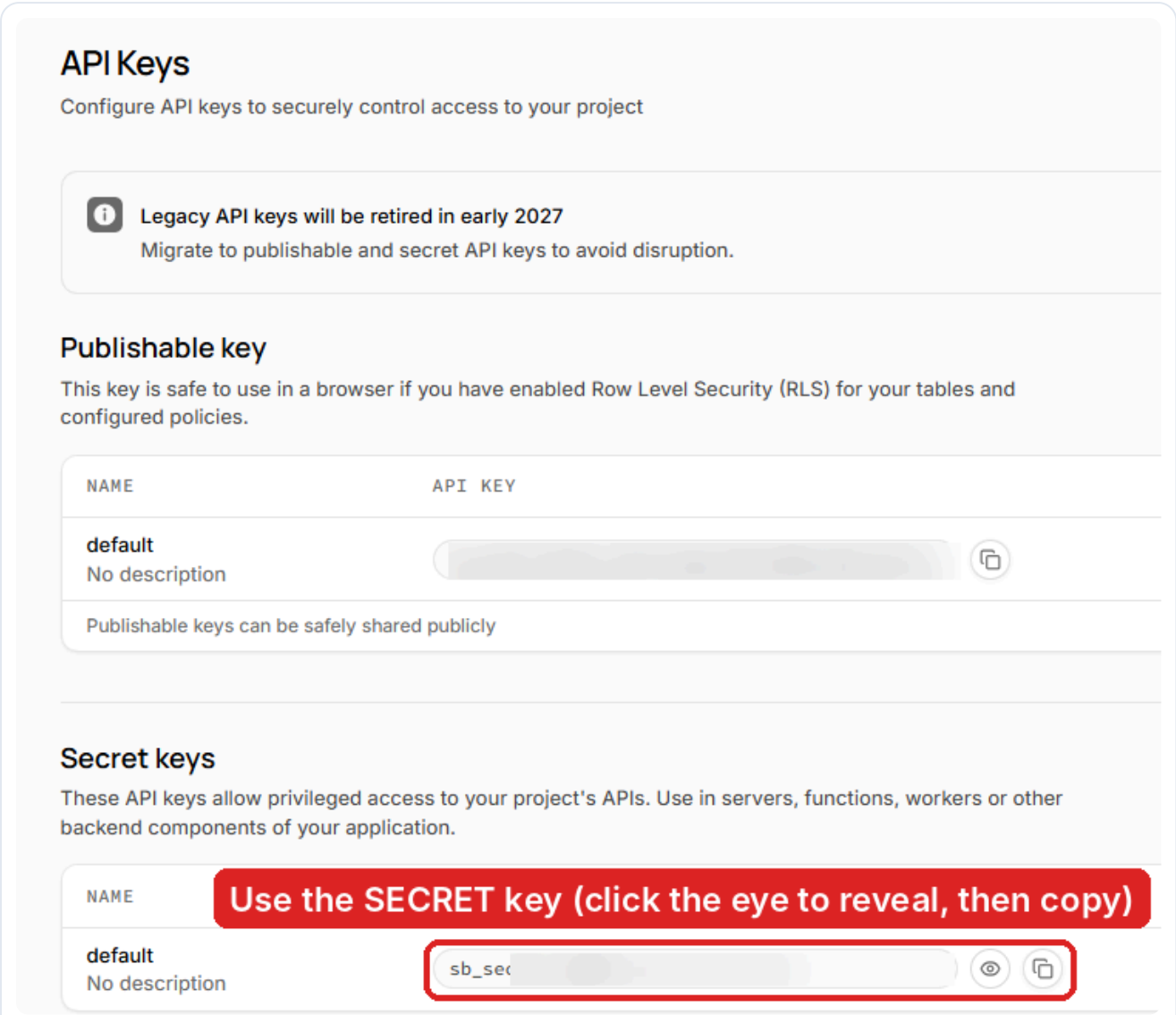
4. Set up the database in Supabase

- Use one Supabase project. Its id in the address bar (`supabase.com/dashboard/project/<project-id>`) must match your `SUPABASE_URL`
- **SQL Editor** → **New query**: run `seed/00_starter_example.sql`, then `seed/campusbite.sql`
- **Table Editor**: `menu_items` has 20 rows

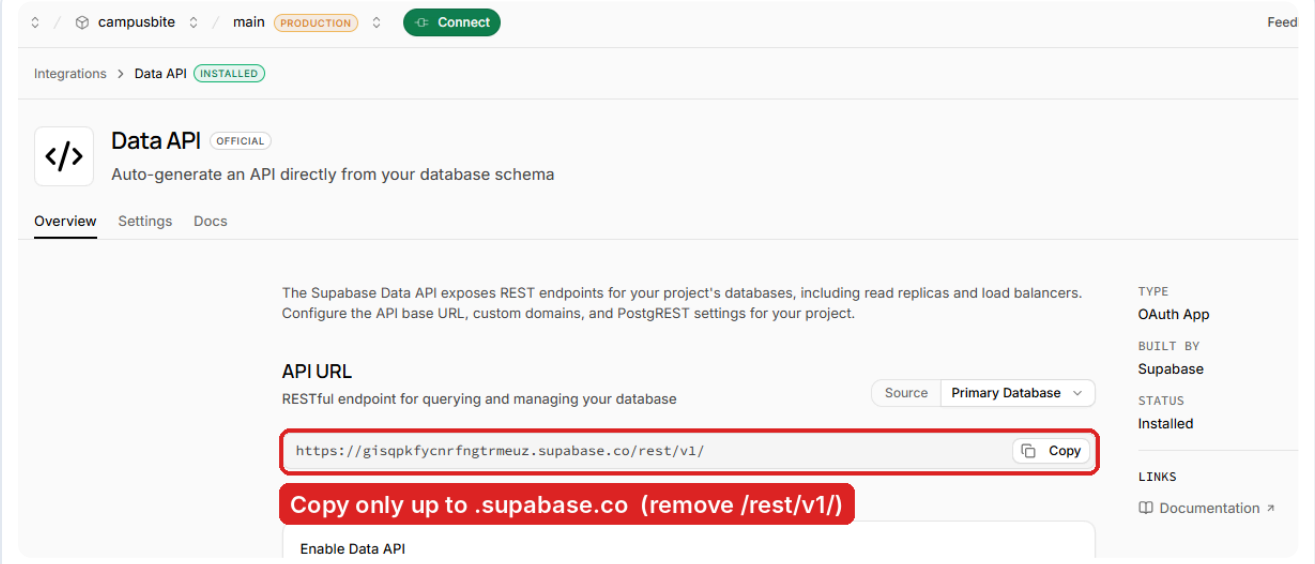


Supabase SQL Editor: click New, paste the whole .sql file, then click Run.

- **Project Settings** → **API Keys**: copy the **secret** key into `.env.local`



Project Settings → API Keys: copy the secret key, not the publishable key.



Your Project URL: copy only up to .supabase.co.

5. Get your Gemini key

- Google AI Studio → **Get API key** → **Create API key**, then paste it into `.env.local`

6. Run the starter

```
TERMINAL

npm run dev
```

- Open `localhost:3000/api/health`: all settings `true`, `databaseCheck: ok`
- Open `localhost:3000`, log in with your passcode, and ask "Which tasks are still open?" You should see **Done · listTasks**

> **Checkpoint:** the health check is all ok and the starter agent answers.

> **Remember:** Next.js reads `.env.local` only when it starts. After any change, press `Ctrl+C` and run `npm run dev` again.

05 Before you prompt: two things to know

1. The rules file

Your repo has a file `.github/copilot-instructions.md`. Copilot reads it automatically with every prompt, so you do not have to repeat the project rules each time. (`CLAUDE.md` is the same file for Claude Code.)

It tells the AI things like:

- The logged-in student is always `S001`. Never take a student id as a tool argument
- Tools live in `lib/agent/tools.ts` and use `defineTool()` with a Zod schema
- Orders must use the database functions `place_order` and `cancel_order`
- Never use the Supabase secret key in components

> **Key idea:** a short rules file gives better code from every prompt. For the hackathon, you will write one for your own problem.

2. Use Copilot Free wisely

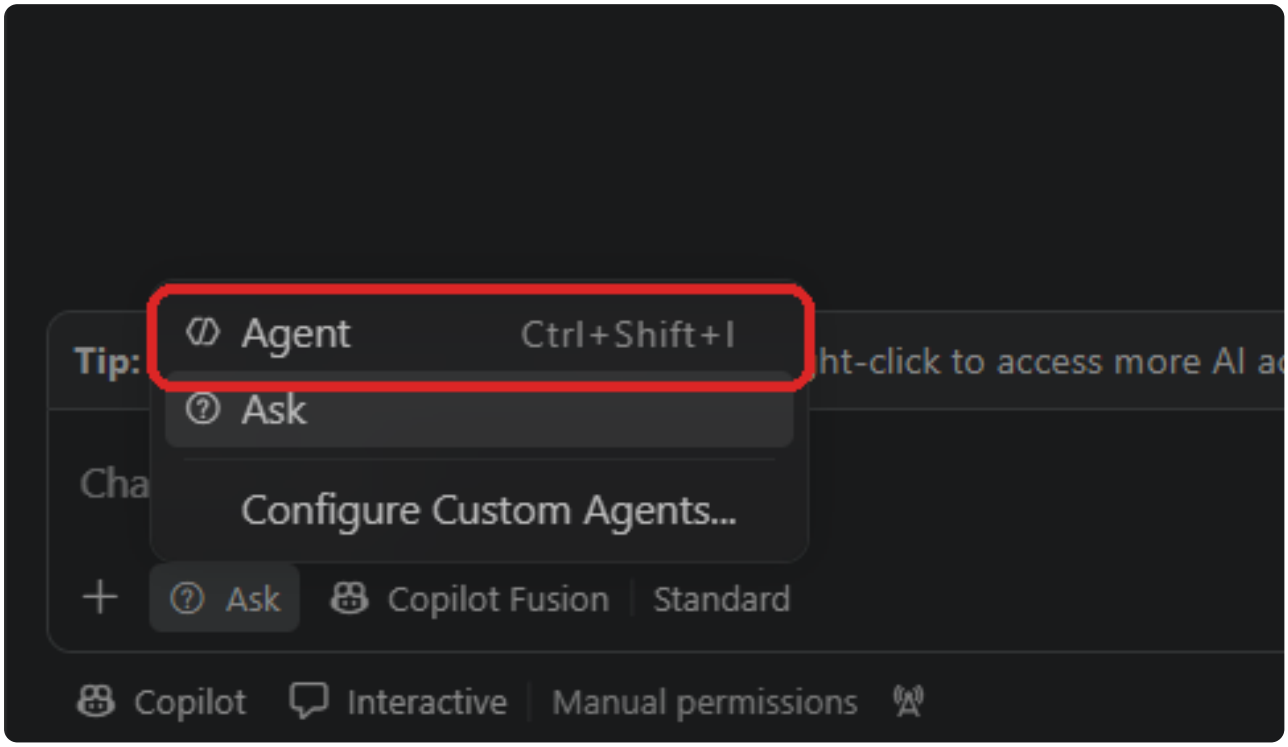
This handbook uses about 12 chat prompts. Copilot Free has a monthly chat limit, so:

- Copy each prompt exactly, in one message. One clear prompt is better than five small ones
- Always attach context (`#file`, `#codebase`) so the first answer is right
- Read the error yourself first, then ask Copilot
- Use inline suggestions (`Tab`) for small edits; they do not use chat requests
- If you reach the limit, ask your team for help reading code and continue with inline suggestions

06 Step 1: Plan with AI

MODE Ask

Open Copilot Chat (`Ctrl+Alt+I`) and set the mode to **Ask**. Ask mode only answers; it does not change files.



The mode menu in Copilot Chat: Ask for planning, Agent for building.

COPILOT PROMPT

```
#codebase Read this Next.js starter and seed/campusbite.sql.
I want to turn this starter into CampusBite, an SRM canteen ordering agent.
Students order food in plain English; the AI agent checks the menu, wallet and pickup slots, then
places the order. A kitchen screen shows live orders.
Give me a short build plan: which files to create or change, which agent tools we need, and which
API routes and pages. Do not write code yet.
```

Check: the plan names six tools, `lib/campusbite.ts`, the API routes and the kitchen page, and says to use the database functions instead of updating the wallet directly.

Key idea: good developers plan before they code, even with AI.

07 Step 2: Read tools (the agent's eyes)

MODE Agent

COPILOT PROMPT

```
In #file:lib/agent/tools.ts replace the 3 task example tools with 4 read tools for CampusBite,
using defineTool like the existing examples.
- Create lib/campusbite.ts with: export const CURRENT_STUDENT_ID = "S001"; and a helper
formatSlot("13:15:00") → "1:15 pm".
- getMenu: reads menu_items where stock > 0. Optional args: category (meal, snack, drink, sweet),
veg_only (boolean), max_price (integer). Order by category, then price. Return id, name,
category, price, is_veg, stock, description.
- getWalletBalance: no args. Returns name and balance for CURRENT_STUDENT_ID from wallets.
- getPickupSlots: no args. Reads pickup_slots, returns slot_id, time (formatted) and free
(capacity - booked), only slots with free > 0.
- getMyOrders: no args. Returns the student's last 10 orders with id, status, total, pickup time
and item names (join order_items and menu_items).
Descriptions must start with "Use this when...". Never take a student id as an argument.
```

Check before Keep: `lib/campusbite.ts` exists; `getMenu` has `.gt("stock", 0)`; no tool takes a student id; the tools array has the 4 tools.

Test in the app: type (do not click the old sample buttons) `What's veg under Rs 80 today?` and `What is my wallet balance?`. Paneer is not listed because it is sold out; the wallet shows Rs 150.

Remember: the screen does not change in this step. Only the agent's tools changed. The screen changes in Step 6.

Key idea: an AI agent is only as good as its tools. Click the grey **Done · getMenu** box: Gemini chose `veg_only` and `max_price` by itself.

08 Step 3: Write tools (the agent's hands)

MODE Agent

COPILOT PROMPT

```
In #file:lib/agent/tools.ts add 2 write tools (writes: true):
- placeOrder: args items (array of { menu_item_id, quantity 1-5 }, at least 1) and slot_id. Call
getSupabase().rpc("place_order", { p_student_id: CURRENT_STUDENT_ID, p_slot_id: slot_id, p_items:
items }). Throw new Error(error.message) on error, else return data.
- cancelOrder: arg order_id. Call rpc("cancel_order", { p_student_id: CURRENT_STUDENT_ID,
p_order_id: order_id }) the same way.
Do not update wallets or stock directly in TypeScript; the database functions do all checks.
The tools array must be exactly: getMenu, getWalletBalance, getPickupSlots, getMyOrders,
placeOrder, cancelOrder.
```

Check before Keep: both tools have `writes: true` and use `rpc(...)`; there is no `.update(...)` on wallets or menu items; all 6 tools are in the array.

Remember: do not test ordering yet. Without the Step 4 system prompt, the agent may guess ids, undo its own order and run out of steps. Go straight to Step 4.

09

Step 4: System prompt (the agent's job description)

MODE Agent

COPILOT PROMPT

```
Rewrite buildSystemPrompt in #file:lib/agent/systemPrompt.ts for CampusBite:
- You are CampusBite, the friendly SRM canteen ordering assistant. The student is
CURRENT_STUDENT_ID.
- Keep the line with today's date. Pickup is today only, 12:00 pm to 2:30 pm.
- Always use tools for menu, prices, stock, wallet and slots. Never invent them.
- Never guess an item id or slot id. Before calling placeOrder, call getMenu and getPickupSlots
in the same turn to get the real ids, even if you listed them earlier.
- Before ordering, check menu, wallet and free slots. If the student gives a budget, fit the
total to it.
- If the student gives a clear request with a budget and a pickup time, choose the best
combination and place the order directly. Only ask when the request is unclear.
- If no pickup time is given, use the earliest free slot. If a time is full, offer the nearest
free time.
- If an item is sold out or the wallet is too low, explain simply and suggest a cheaper or
similar item.
- If the student picks an option you offered (for example "option 1"), place that order directly.
- Only call cancelOrder when the student clearly asks to cancel. Never cancel an order to fix
your own mistake; explain the problem instead.
- If placeOrder fails, read the error, fix the ids and try once more, then explain to the
student.
- After ordering, reply with items, total, pickup time and wallet balance left. Prices like "Rs
70".
- If the kitchen has started an order, explain it cannot be cancelled.
- Keep replies short. Only help with the canteen.
```

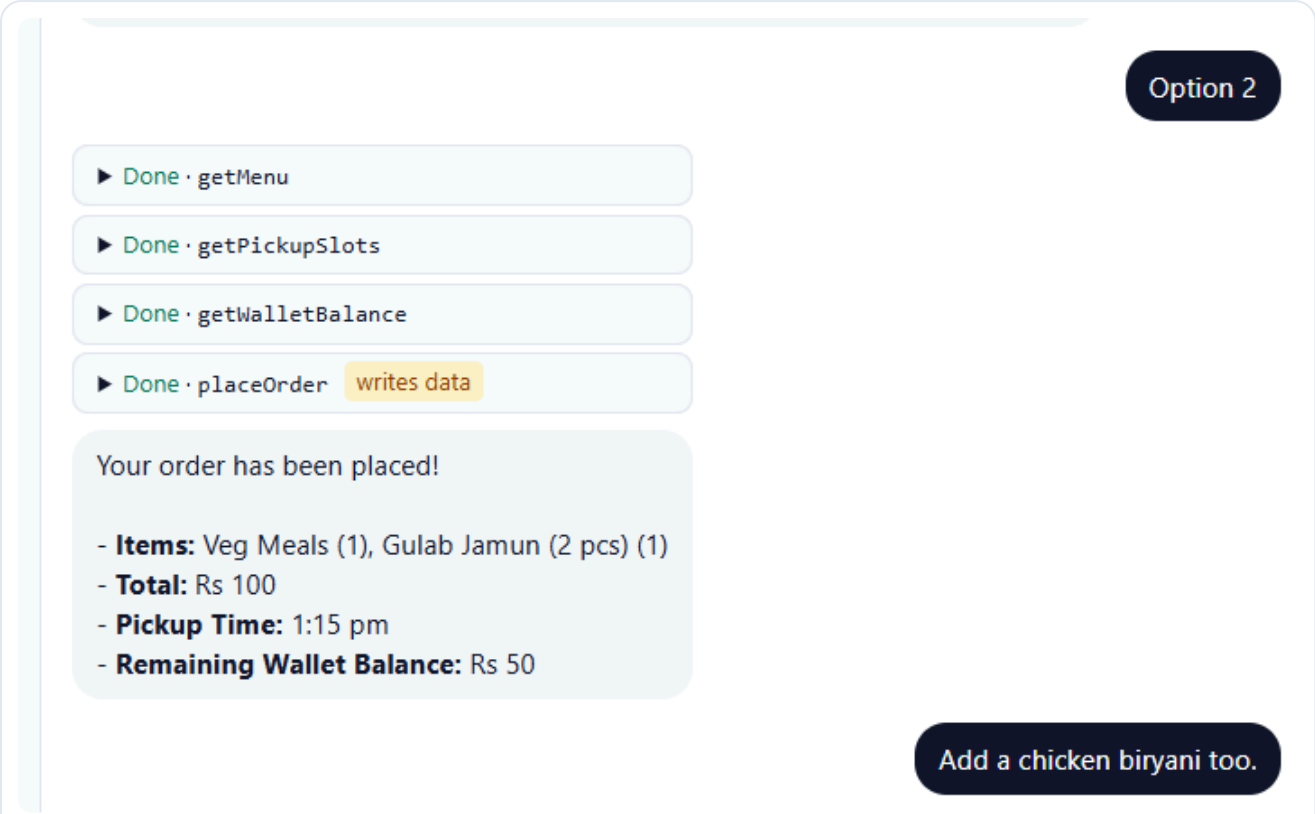
Then give the agent a little more room:

COPILOT PROMPT

```
In #file:lib/agent/runAgent.ts change MAX_ROUNDS from 5 to 8.
```

Test in the app: refresh the page for a fresh chat, then type `I have Rs 150. Get me lunch with something sweet, pick up at 1:15.` You should see **getMenu** → **getWalletBalance** → **getPickupSlots** → **placeOrder** (yellow "writes data" tag), Rs 100, 1:15 pm and Rs 50 left.

Test in the app: `Add a chicken biryani too.` The agent refuses (Rs 120 is more than Rs 50) and suggests cheaper items.



Four tools in one request: the order is placed and the wallet drops to Rs 50.

Key idea: the system prompt is the agent's job description. One sentence made the agent use four tools and change three tables. That is an agent, not a chatbot.

Checkpoint: save your progress. In **Source Control** (`Ctrl+Shift+G`), click the **sparkle icon** in the message box and Copilot writes the commit message for you. Read it, edit if needed, then click **Commit**. Or in the terminal: `git add .` then `git commit -m "CampusBite agent: 6 tools and system prompt"` .

10 Step 5: API routes

MODE Agent (Copilot), or Claude Code if you have it

API routes are the "waiters" that carry data between the screens and the database.

COPILOT PROMPT

```
Create 3 API routes, following the pattern of #file:app/api/tasks/route.ts:
1. app/api/menu/route.ts GET: all menu_items (including sold out), ordered by category and price.
2. app/api/me/route.ts GET: wallet of CURRENT_STUDENT_ID and their last 5 orders with pickup time
and item names.
3. app/api/kitchen/route.ts GET: orders with status placed, preparing or ready, with student
name, pickup time and items, oldest first.
   POST { order_id, status }: validate with Zod. Only allow the next step: placed → preparing →
ready → collected. Return 409 for any other change.
Add export const dynamic = "force-dynamic" to each. Log errors with console.error.
Also update app/api/health/route.ts to check the menu_items table instead of tasks.
```

Test in the app: open `localhost:3000/api/menu` (20 items, Paneer `stock: 0`), `/api/me` (your balance and order) and `/api/kitchen` (your order plus 2 from other students).

11 Step 6: Student screen

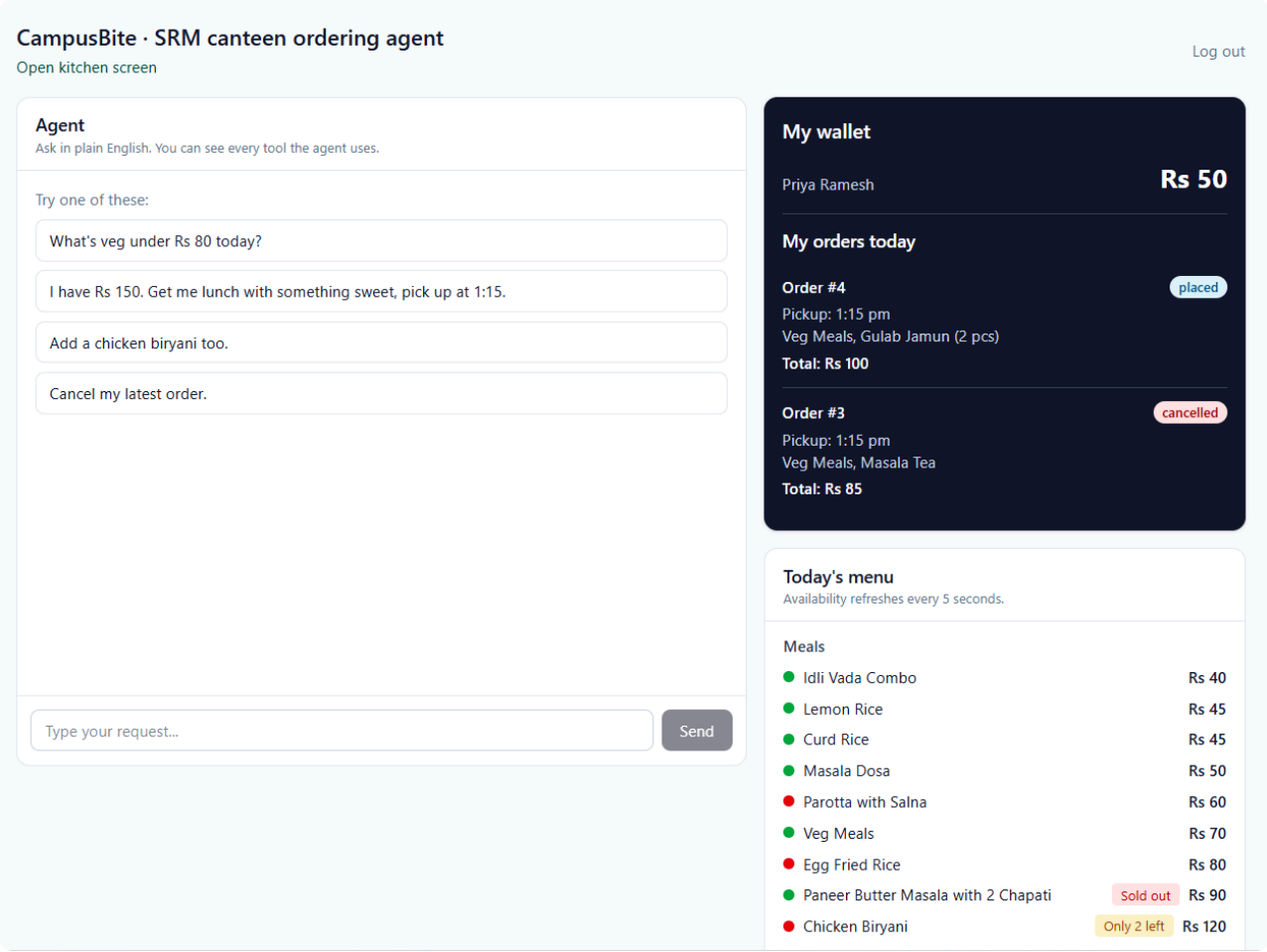
MODE Agent (Copilot), or Claude Code if you have it

COPILOT PROMPT

```
Build the CampusBite student screen:
1. components/WalletOrders.tsx: a dark card showing the wallet balance (Rs) and student name,
then "My orders today" with order number, pickup time, items, total and a coloured status badge.
Loads /api/me.
2. components/MenuPanel.tsx: today's menu grouped by meals, snacks, sweets, drinks. Each row:
green or red veg dot, name, stock ("Sold out" red badge at 0, "Only N left" amber badge at 3 or
less), price. Loads /api/menu.
3. Both reload when refreshKey changes AND every 5 seconds, so kitchen updates appear.
4. In app/page.tsx replace TaskList with WalletOrders above MenuPanel on the right. Title
"CampusBite · SRM canteen ordering agent", and an "Open kitchen screen" link to /kitchen.
5. In components/AgentChat.tsx set SAMPLE_PROMPTS to: "What's veg under Rs 80 today?", "I have Rs
150. Get me lunch with something sweet, pick up at 1:15.", "Add a chicken biryani too.", "Cancel
my latest order."
6. Supabase may return a related row as an object or a one-item array; handle both so item names
and pickup times always show.
Use Tailwind. Delete components/TaskList.tsx and app/api/tasks.
```

Test in the app: refresh `localhost:3000`. You see the CampusBite header, the wallet card with your balance, your order with a "placed" badge, and the menu with Paneer "Sold out" and Chicken Biryani "Only 2 left".

Remember: the "Open kitchen screen" link shows a 404 until Step 7. That is expected.



The finished student screen: agent chat, wallet, my orders and the live menu.

Key idea: we built the brain first, then the face. Every number on this screen came from the agent's work.

12 Step 7: Kitchen screen

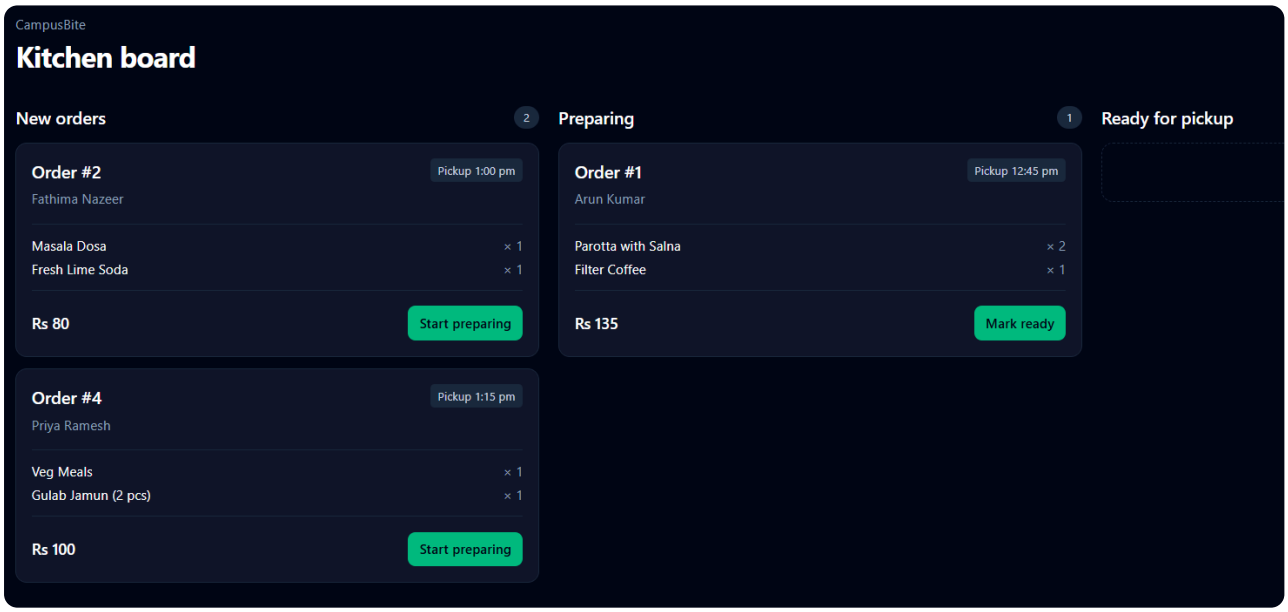
MODE | Agent (Copilot), or Claude Code if you have it

COPILOT PROMPT

```
Create app/kitchen/page.tsx: a dark full-screen kitchen board with 3 columns: New orders, Preparing, Ready for pickup.
Each order card shows order number, pickup time, student name, items and total, with one button: "Start preparing" (placed), "Mark ready" (preparing), "Collected" (ready). The button calls POST /api/kitchen.
Reload /api/kitchen every 3 seconds. Show a green "Live" badge in the header. Handle related rows that may be an object or an array. Use Tailwind.
```

Check: the file must be exactly `app/kitchen/page.tsx`. If `/kitchen` still shows 404, restart with `Ctrl+C` and `npm run dev`.

Test in the app: put the student screen and `/kitchen` side by side. Your order appears under **New orders** next to the other students' orders.



The kitchen board: New orders, Preparing and Ready for pickup.

13 Step 8: Build, commit and deploy

MODE Ask

In the terminal, stop the app (`Ctrl+C`) and run `npm run build`. If there are errors, send:

COPILOT PROMPT

```
#terminalLastCommand Explain these errors in one line each and tell me exactly what to change in which file.
```

Fix them and run `npm run build` again until you see **Compiled successfully**.

Commit and push:

TERMINAL

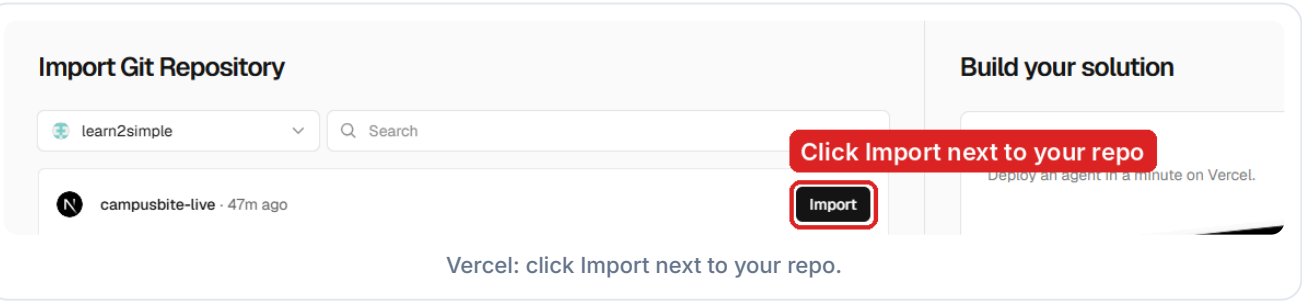
```
git status
git add .
git commit -m "CampusBite: agent, APIs, student and kitchen screens"
git push
```

You can also commit from **Source Control** with the **sparkle icon** for the message, then click **Sync Changes** to push.

> **Check:** in the `git status` list (or the Source Control list), `.env.local` must not appear, and `.env.example` must not show as changed.

Deploy on Vercel:

1. Vercel → **Add New** → **Project** → import your `campusbite` repo



Vercel: click Import next to your repo.

2. **Environment Variables**: open `.env.local`, select all, copy, and paste into the first **Key** box. Vercel creates all 5 rows. Mark `APP_PASSCODE`, `GEMINI_API_KEY` and `SUPABASE_SECRET_KEY` as **Secret**. Tick **Production** and **Preview**
3. Click **Deploy** and wait for **Ready**
4. Open your link and check `/api/health`, the CampusBite screen and `/kitchen`

> **Remember:** after changing a variable on Vercel, save it and **Redeploy**. If others see a Vercel login page, turn off **Vercel Authentication** under **Settings** → **Deployment Protection**.

Checkpoint: your live link shows CampusBite and an order works through the chat.

14 Full test: six moments

Run `seed/campusbite.sql` again in Supabase for fresh data. Open the student screen on the left and `/kitchen` on the right.

#	Type or do	What you should see	What it shows
1	"What's veg under Rs 80 today?"	`getMenu` with filters; Paneer is missing	The agent picks the tool and filters itself
2	"I have Rs 150. Get me lunch with something sweet, pick up at 1:15."	Four tools; wallet drops to Rs 50; order appears in the kitchen	One sentence, four tools, three tables changed
3	"Add a chicken biryani too."	Refused: Rs 120 is more than Rs 50; cheaper suggestions	Rules in the database cannot be broken
4	Kitchen: **Start preparing**	Student badge turns "preparing" within 5 seconds	Two screens share live data
5	"Cancel my latest order."	Refused: the kitchen has already started	Business rules protect the canteen too
6	"Order a filter coffee for 2 pm." then "Cancel it."	Order placed, then cancelled with a refund; stock goes back	Every write is undone correctly

Tip: click a grey tool step in the chat to see the arguments Gemini sent and the result your code returned. This is function calling in action.

More to try: "Can I pick up at 12:30?" (full slot) · "Get me 6 samosas" (5-item limit) · "Write me a poem" (the agent stays on topic).

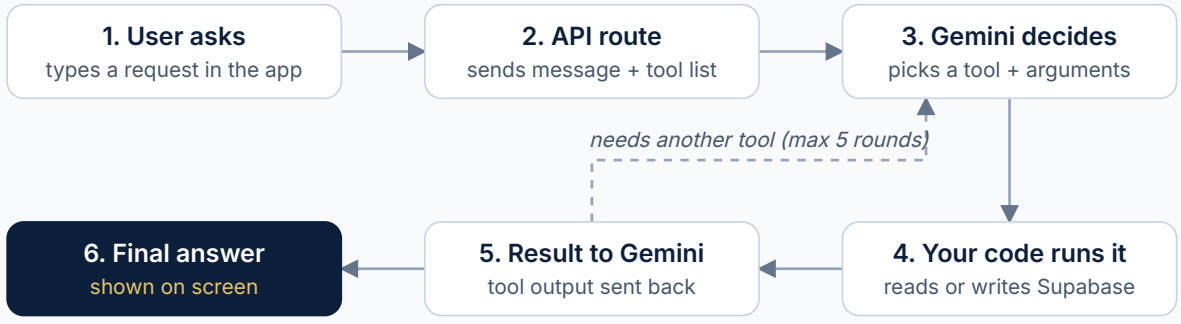
Reset any time: run `seed/campusbite.sql` again. It recreates all CampusBite tables with fresh stock, wallets and slots.

15 Understand the agent loop

Judges on Day 2 will ask: "How does your agent work?" Every team member should be able to answer.

1. Open `lib/agent/runAgent.ts` and select the `for` loop
2. In Copilot Chat (**Ask** mode), type `/explain` and send
3. In your own words, write 3 lines in your `README.md` under a heading "How the agent works"

Check: your 3 lines should cover: Gemini receives the message and the tool list; Gemini asks for a tool and our code runs it after a Zod check; the result goes back to Gemini until it gives a final answer (at most 8 rounds).



The agent loops between Gemini and your tools until it can answer.

16 Stretch features

Finished early? Pick one or more. Use **Agent** mode, read every change, then test.

1. Reorder my usual

COPILOT PROMPT

```
In #file:lib/agent/tools.ts add a tool reorderLast (writes: true, no args). It reads the student's latest order that is not cancelled (with its items), picks the earliest free pickup slot, and calls rpc("place_order") with the same items. Description: "Use this when the student asks to order the same as last time." Add it to the tools array.
```

Test in the app: "Order what I had last time."

2. Daily spending limit of Rs 200

COPILOT PROMPT

```
In #file:lib/agent/tools.ts, before placeOrder calls rpc("place_order"), work out today's total for CURRENT_STUDENT_ID (sum of orders created today that are not cancelled) and the new order total from menu_items prices. If the sum would go above 200, throw new Error("Daily limit of Rs 200 reached. You have Rs X left to spend today."). Add the rule to the system prompt too.
```

Test in the app: place orders until the total passes Rs 200. The agent should explain the limit.

3. Kitchen "Sold out" button

COPILOT PROMPT

```
Add a "Menu stock" panel at the bottom of app/kitchen/page.tsx that lists menu items with their stock and a "Mark sold out" button. Create app/api/kitchen/stock/route.ts with a POST { menu_item_id } that sets stock to 0 (validate with Zod). The student menu and the agent must stop offering that item.
```

> **Test in the app:** mark Masala Dosa sold out in the kitchen, then ask the agent for a dosa.

4. Veg-only preference

COPILOT PROMPT

```
I want a saved veg-only preference per student.
1. Give me SQL to add a column veg_only boolean not null default false to the wallets table (I will run it in Supabase).
2. Add a tool setVegOnly (writes: true, arg veg_only boolean) that updates it for CURRENT_STUDENT_ID.
3. Make getMenu return only veg items when the student's veg_only is true.
4. Add the preference to the system prompt.
```

> **Test in the app:** "I am vegetarian, remember that." Then "What's for lunch under Rs 100?"

5. Mobile check

COPILOT PROMPT

```
Review #file:app/page.tsx and the WalletOrders and MenuPanel components for a 360px wide phone screen. Fix anything that overflows or is hard to tap. Do the same for app/kitchen/page.tsx.
```

> **Test in the app:** in Chrome press `F12`, click the phone icon, and choose a phone size.

> **Remember:** commit after each feature that works, so you can go back if the next one breaks something.

17 From CampusBite to your hackathon

Tomorrow your team builds a new agent app for your problem statement. Use the same path you followed today.

Today (CampusBite)	Tomorrow (your problem)
Step 1: plan with AI	Plan your app with the prompt below
Rules file	Write `.github/copilot-instructions.md` for your problem
Steps 2–3: read tools, then write tools	Design your tools with the table below
Step 4: system prompt	Write your agent's job description
Steps 5–7: API routes and screens	Build the screens your users need
Step 8: build and deploy	Deploy by Checkpoint 1, then keep pushing

1. Planning prompt (Ask mode, after you run your seed file)

COPILOT PROMPT

```
#codebase Read this starter and seed/<your seed file>.sql.
Our problem: <business user> has this problem: <problem in one line>.
Our agent must: <what it should do, in 2 lines>.
Give a short build plan: the read tools and write tools we need (name, arguments, which table),
the business rules to check in code, the API routes and the pages. Do not write code yet.
```

2. Tool design table (fill it in as a team before coding)

Tool name	Reads or writes	Arguments	Table	Business rule checked
	Reads			
	Reads			
	Writes			

3. Rules file starter for `.github/copilot-instructions.md`

FILE CONTENT

```
# <App name> project rules
- App: <one line about the app and its user>. The logged-in user is always "<id>". Never take a
user id as a tool argument.
- Stack: Next.js App Router, TypeScript, Tailwind CSS, Supabase, Gemini function calling, Zod.
- Agent tools live in lib/agent/tools.ts and use defineTool() with a Zod schema.
- Tables: <list your tables>.
- Business rules: <your rules>. Check them in code and throw new Error("clear message") when a
rule fails.
- Only lib/ and app/api/ may call getSupabase(). Never use the secret key in components.
```

> **Key idea:** the order matters. Read tools first, then write tools, then the system prompt, then screens. A working agent in the chat is worth more marks than a pretty screen with no agent.

18 Common errors and fixes

Problem	Cause	Fix
`APP_PASSCODE is not set on the server`	Values typed into `.env.example`, or the app not restarted	Put values in `.env.local`, save, `Ctrl+C`, `npm run dev`
"Could not find the table ... in the schema cache"	`.env.local` points to a different Supabase project	Compare `supabaseProject` in `/api/health` with your dashboard address; run the seed files in that project
`Invalid API key` (Supabase)	Secret key from another project, or the anon / publishable key	Copy the **secret** key from the same project, then restart
"Invalid path specified in request URL"	Extra text after `.supabase.co`	Keep only `https://<project-id>.supabase.co`
Gemini "model is no longer available" (404)	Old model name	`GEMINI_MODEL=gemini-3.8-flash`, restart
Gemini `429` or quota	Too many requests	Wait one minute; use short prompts
Agent guesses ids, cancels its own order, "too many steps"	System prompt not updated	Use the full Step 4 prompt and `MAX_ROUNDS = 8`
Screen still shows "Hackathon Starter"	Expected before Step 6	The screen changes in Step 6
`/kitchen` shows 404	Kitchen page not built yet or wrong path	Do Step 7; the file must be `app/kitchen/page.tsx`
`Push cannot contain secrets` (GH013)	Real keys in `.env.example`	Do not click "allow the secret". Empty the values in `.env.example`, then run `git add .env.example`, `git commit --amend --no-edit` and `git push`. If the key is in an older commit, ask the trainer.
`git push` 403 permission denied	VS Code signed in to a different GitHub account	Accounts icon → sign out → sign in with the account that owns the repo
Live site shows the old starter	Vercel built an old commit	Push your latest commit, wait for **Ready**, then `Ctrl+F5`
Copilot changed files you did not want	–	Click **Undo** in the chat, or Source Control → right-click → **Discard Changes**

Still stuck after 10 minutes? Raise your hand. Show the exact error message and the step number.

19 Key ideas to remember

- Good developers plan before they code, even with AI.
- An AI agent is only as good as its tools. Read tools are its eyes; write tools are its hands.
- The system prompt is the agent's job description.
- Gemini never touches the database. It asks for a tool, and your code does the work.
- Business rules live in the database, so the AI cannot break them.
- You own every line, even if AI wrote it. Read before you click **Keep**.
- Keys go only in `.env.local`. Never in `.env.example`, never in chat, never in frontend code.
- A short rules file makes every AI prompt better.
- Build in order: read tools, write tools, system prompt, then screens.

20 Files and links

Item	Where
Starter template	`github.com/learn2simple/campusbite-live` → **Use this template**
Database setup and reset	`seed/campusbite.sql`
All prompts in one file	`live-demo/LIVE_BUILD_PROMPTS.md`
Full test script	`live-demo/DEMO_SCRIPT.md`
AI project rules (read automatically)	`.github/copilot-instructions.md` (Copilot), `CLAUDE.md` (Claude Code)
Related guides	Guide 1: Setup Guide · Guide 3: Hackathon Guide